

VELORA — Laravel 13 Enterprise Setup Guide (Part 2)

Steps 10–18: Service Layer, Helpers, Controllers, Routes, Middleware, Config, Logging, DB

STEP 10 — Setup Service Layer

Abstract Base Service — `app/Shared/Services/BaseService.php`

```
<?php

namespace App\Shared\Services;

abstract class BaseService
{
    // Semua Service extend class ini
    // Bisa tambahkan shared logic: logging, event dispatching, dll
}
```

Contoh Service —

app/Modules/Tenant/Services/BusinessService.php

```
<?php

namespace App\Modules\Tenant\Services;

use App\Modules\Tenant\DTOs\CreateBusinessDTO;
use App\Modules\Tenant\Events\BusinessCreated;
use App\Modules\Tenant\Models\Business;
use App\Modules\Tenant\Repositories\Contracts\BusinessRepositoryInterface;
use App\Shared\Exceptions\VeloraException;
use App\Shared\Services\BaseService;
use Illuminate\Pagination\LengthAwarePaginator;
use Illuminate\Support\Facades\DB;

class BusinessService extends BaseService
{
    public function __construct(
        private readonly BusinessRepositoryInterface $businessRepository
    ) {}

    public function getAll(array $filters = []): LengthAwarePaginator
    {
        return $this->businessRepository->all($filters);
    }

    public function create(CreateBusinessDTO $dto): Business
    {
        return DB::transaction(function () use ($dto) {
            $business = $this->businessRepository->create($dto->toArray());
            event(new BusinessCreated($business));
            return $business;
        });
    }

    public function findOrFail(string $id): Business
    {
        $business = $this->businessRepository->findById($id);

        if (!$business) {
            throw VeloraException::notFound('Business');
        }
    }
}
```

```
        return $business;
    }
}
```

DTO Example — app/Modules/Tenant/DTOs/CreateBusinessDTO.php

```
<?php

namespace App\Modules\Tenant\DTOs;

class CreateBusinessDTO
{
    public function __construct(
        public readonly string $name,
        public readonly string $email,
        public readonly string $phone,
        public readonly string $ownerId,
        public readonly ?string $address = null,
        public readonly string $currency = 'IDR',
        public readonly string $timezone = 'Asia/Jakarta',
    ) {}

    public static function fromRequest(array $validated): static
    {
        return new static(
            name:      $validated['name'],
            email:     $validated['email'],
            phone:     $validated['phone'],
            ownerId:   $validated['owner_id'],
            address:   $validated['address'] ?? null,
            currency:  $validated['currency'] ?? 'IDR',
            timezone:  $validated['timezone'] ?? 'Asia/Jakarta',
        );
    }

    public function toArray(): array
    {
        return [
            'name'      => $this->name,
            'email'     => $this->email,
            'phone'     => $this->phone,
            'owner_id' => $this->ownerId,
            'address'   => $this->address,
            'currency'  => $this->currency,
            'timezone'  => $this->timezone,
        ];
    }
}
```

}

}

STEP 11 — Setup Global Helpers

app/Shared/Helpers/MoneyHelper.php

```
<?php

namespace App\Shared\Helpers;

class MoneyHelper
{
    public static function format(int $amount, string $currency = 'IDR'): string
    {
        return match ($currency) {
            'IDR' => 'Rp ' . number_format($amount, 0, ',', '.'),
            'USD' => '$' . number_format($amount / 100, 2),
            default => number_format($amount, 0, ',', '.') . ' ' . $currency,
        };
    }

    public static function toSmallestUnit(float $amount, string $currency = 'IDR'): int
    {
        return match ($currency) {
            'IDR' => (int) round($amount),
            'USD' => (int) round($amount * 100),
            default => (int) round($amount),
        };
    }

    public static function fromSmallestUnit(int $amount, string $currency = 'IDR'): float
    {
        return match ($currency) {
            'IDR' => (float) $amount,
            'USD' => $amount / 100,
            default => (float) $amount,
        };
    }
}
```

app/Shared/Helpers/CodeGenerator.php

```
<?php
```

```
namespace App\Shared\Helpers;
```

```
class CodeGenerator
```

```
{
```

```
    public static function sku(string $prefix = 'SKU'): string
```

```
    {
```

```
        return strtoupper($prefix) . '-' . strtoupper(substr(uniqid(), -6));
```

```
    }
```

```
    public static function invoiceNumber(string $prefix = 'INV'): string
```

```
    {
```

```
        return $prefix . '/' . date('Ymd') . '/' . str_pad(random_int(1, 9999), 4, '0', STR_PAD_
```

```
    }
```

```
    public static function orderNumber(): string
```

```
    {
```

```
        return 'ORD' . date('YmdHis') . str_pad(random_int(0, 999), 3, '0', STR_PAD_LEFT);
```

```
    }
```

```
}
```


app/Shared/Helpers/helpers.php (Global Functions)

```
<?php
```

```
use App\Shared\Helpers\ApiResponse;
```

```
use App\Shared\Helpers\MoneyHelper;
```

```
if (!function_exists('api_success')) {  
    function api_success(mixed $data = null, string $message = 'Success', int $code = 200) {  
        return ApiResponse::success($data, $message, $code);  
    }  
}
```

```
if (!function_exists('api_error')) {  
    function api_error(string $message, int $code = 400, string $errorCode = '') {  
        return ApiResponse::error($message, $code, $errorCode);  
    }  
}
```

```
if (!function_exists('api_created')) {  
    function api_created(mixed $data, string $message = 'Created successfully') {  
        return ApiResponse::created($data, $message);  
    }  
}
```

```
if (!function_exists('api_paginated')) {  
    function api_paginated(\Illuminate\Pagination\LengthAwarePaginator $paginator, mixed $data,  
        return ApiResponse::paginated($paginator, $data, $message);  
    }  
}
```

```
if (!function_exists('format_money')) {  
    function format_money(int $amount, string $currency = 'IDR'): string {  
        return MoneyHelper::format($amount, $currency);  
    }  
}
```

```
if (!function_exists('tenant_id')) {  
    function tenant_id(): ?string {  
        return auth()->user()->business_id;  
    }  
}
```


STEP 12 — Setup Base Controller

app/Shared/Controllers/BaseController.php

```
<?php

namespace App\Shared\Controllers;

use App\Shared\Helpers\ApiResponse;
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;
use Illuminate\Foundation\Validation\ValidatesRequests;
use Illuminate\Pagination\LengthAwarePaginator;
use Illuminate\Routing\Controller;

abstract class BaseController extends Controller
{
    use AuthorizesRequests, ValidatesRequests;

    protected function success(mixed $data = null, string $message = 'Success', int $code = 200)
    {
        return ApiResponse::success($data, $message, $code);
    }

    protected function created(mixed $data, string $message = 'Created successfully')
    {
        return ApiResponse::created($data, $message);
    }

    protected function paginated(LengthAwarePaginator $paginator, mixed $data, string $message = '')
    {
        return ApiResponse::paginated($paginator, $data, $message);
    }

    protected function error(string $message, int $code = 400, string $errorCode = '')
    {
        return ApiResponse::error($message, $code, $errorCode);
    }

    protected function notFound(string $message = 'Resource not found')
    {
        return ApiResponse::notFound($message);
    }
}
```

```

        protected function noContent()
        {
            return ApiResponse::noContent();
        }
    }
}

```

Contoh Module Controller

```
<?php
```

```
namespace App\Modules\Tenant\Controllers;
```

```

use App\Modules\Tenant\DTOs\CreateBusinessDTO;
use App\Modules\Tenant\Requests\StoreBusinessRequest;
use App\Modules\Tenant\Resources\BusinessResource;
use App\Modules\Tenant\Services\BusinessService;
use App\Shared\Controllers\BaseController;

```

```

class BusinessController extends BaseController
{
    public function __construct(private readonly BusinessService $service) {}

    public function index()
    {
        $paginator = $this->service->getAll(request()->all());
        return $this->paginated($paginator, BusinessResource::collection($paginator), 'Business');
    }

    public function store(StoreBusinessRequest $request)
    {
        $dto = CreateBusinessDTO::fromRequest($request->validated());
        $business = $this->service->create($dto);
        return $this->created(new BusinessResource($business));
    }

    public function show(string $id)
    {
        $business = $this->service->findOrFail($id);
        return $this->success(new BusinessResource($business));
    }
}

```

STEP 13 — Setup API Route Structure

routes/api.php — Master Router

```
<?php
```

```
use Illuminate\Support\Facades\Route;
```

```
Route::prefix('v1')->name('v1.')->group(function () {
```

```
    // Health check
```

```
    Route::get('/health', fn() => api_success(['status' => 'ok', 'version' => config('velora.ver
```

```
    // Module routes (auto-load)
```

```
    $modules = [
```

```
        'Auth', 'Tenant', 'Subscription', 'POS',
```

```
        'Inventory', 'Sales', 'Purchasing', 'Finance',
```

```
        'CRM', 'Employee', 'Notification', 'Report',
```

```
        'Audit', 'Setting', 'AI',
```

```
    ];
```

```
    foreach ($modules as $module) {
```

```
        $routeFile = app_path("Modules/{$module}/Routes/api.php");
```

```
        if (file_exists($routeFile)) {
```

```
            require $routeFile;
```

```
        }
```

```
    }
```

```
});
```

Contoh Module Route — app/Modules/Tenant/Routes/api.php

```
<?php
```

```
use App\Modules\Tenant\Controllers\BusinessController;
```

```
use Illuminate\Support\Facades\Route;
```

```
// Public
```

```
Route::post('/register', [BusinessController::class, 'register'])->name('business.register');
```

```
// Protected
```

```
Route::middleware(['auth:sanctum', 'tenant.scope'])->group(function () {
```

```
    Route::apiResource('businesses', BusinessController::class);
```

```
    Route::apiResource('businesses.branches', BranchController::class);
```

```
});
```

STEP 14 — Setup Middleware Structure

app/Shared/Middleware/EnsureTenantScope.php

```
<?php

namespace App\Shared\Middleware;

use App\Shared\Helpers\ApiResponse;
use Closure;
use Illuminate\Http\Request;

class EnsureTenantScope
{
    public function handle(Request $request, Closure $next)
    {
        $user = $request->user();

        if (!$user || !$user->business_id) {
            return ApiResponse::forbidden('No business context found.', 'NO_TENANT');
        }

        // Share business_id ke seluruh request lifecycle
        app()->instance('current.business_id', $user->business_id);

        return $next($request);
    }
}
```

app/Shared/Middleware/EnsureActiveSubscription.php

```
<?php

namespace App\Shared\Middleware;

use App\Shared\Helpers\ApiResponse;
use Closure;
use Illuminate\Http\Request;

class EnsureActiveSubscription
{
    public function handle(Request $request, Closure $next)
    {
        $user = $request->user();
        $business = $user?->business;

        if (!$business || !$business->hasActiveSubscription()) {
            return ApiResponse::forbidden(
                'Your subscription has expired. Please renew to continue.',
                'SUBSCRIPTION_EXPIRED'
            );
        }

        return $next($request);
    }
}
```

Daftarkan Middleware — bootstrap/app.php

```
->withMiddleware(function (Middleware $middleware) {
    $middleware->statefulApi();

    $middleware->alias([
        'tenant.scope' => \App\Shared\Middleware\EnsureTenantScope::class,
        'subscription.active' => \App\Shared\Middleware\EnsureActiveSubscription::class,
        'branch.access' => \App\Shared\Middleware\EnsureBranchAccess::class,
        'feature.limit' => \App\Shared\Middleware\CheckFeatureLimit::class,
    ]);
})
```


STEP 15 — Setup Config Structure

config/velora.php

```
<?php
```

```
return [  
    'version'          => env('VELORA_VERSION', '1.0.0'),  
    'max_upload_size'  => env('VELORA_MAX_UPLOAD_SIZE', 10240), // KB  
    'default_currency' => 'IDR',  
    'default_timezone' => 'Asia/Jakarta',  
    'pagination'       => [  
        'per_page'      => 15,  
        'max_per_page'  => 100,  
    ],  
    'features' => [  
        'ai_insights'    => env('FEATURE_AI_INSIGHTS', false),  
        'multi_currency' => env('FEATURE_MULTI_CURRENCY', true),  
        'reverb'         => env('FEATURE_REVERB', false),  
    ],  
];
```

config/subscription.php

```
<?php

return [
    'plans' => [
        'starter' => [
            'max_branches'      => 1,
            'max_products'     => 100,
            'max_employees'     => 5,
            'max_customers'     => 500,
            'has_ai'            => false,
            'has_multi_currency' => false,
            'has_report_export' => false,
        ],
        'growth' => [
            'max_branches'      => 3,
            'max_products'     => 1000,
            'max_employees'     => 20,
            'max_customers'     => 5000,
            'has_ai'            => false,
            'has_multi_currency' => true,
            'has_report_export' => true,
        ],
        'enterprise' => [
            'max_branches'      => -1, // unlimited
            'max_products'     => -1,
            'max_employees'     => -1,
            'max_customers'     => -1,
            'has_ai'            => true,
            'has_multi_currency' => true,
            'has_report_export' => true,
        ],
    ],
    'trial_days'      => 14,
    'grace_days'      => 3,
    'currency'        => 'IDR',
];
```

config/pos.php

```
<?php
```

```
return [  
    'receipt' => [  
        'footer_text'    => 'Terima kasih telah berbelanja!',  
        'show_tax'       => true,  
        'show_discount'  => true,  
    ],  
    'tax' => [  
        'default_rate'   => 11, // PPN 11%  
        'inclusive'      => false,  
    ],  
    'rounding' => [  
        'enabled'        => true,  
        'nearest'        => 100, // Round ke 100 IDR  
    ],  
    'shift' => [  
        'require_opening_cash' => true,  
    ],  
];
```

STEP 16 — Setup Logging Structure

config/logging.php — Tambah channels

```
'channels' => [  
  
    'stack' => [  
        'driver' => 'stack',  
        'channels' => ['daily', 'stderr'],  
        'ignore_exceptions' => false,  
    ],  
  
    'daily' => [  
        'driver' => 'daily',  
        'path' => storage_path('logs/velora.log'),  
        'level' => env('LOG_LEVEL', 'debug'),  
        'days' => 30,  
    ],  
  
    'audit' => [  
        'driver' => 'daily',  
        'path' => storage_path('logs/audit.log'),  
        'level' => 'info',  
        'days' => 90,  
    ],  
  
    'api' => [  
        'driver' => 'daily',  
        'path' => storage_path('logs/api.log'),  
        'level' => 'info',  
        'days' => 14,  
    ],  
  
    'stderr' => [  
        'driver' => 'monolog',  
        'handler' => \Monolog\Handler\StreamHandler::class,  
        'formatter' => \Monolog\Formatter\JsonFormatter::class,  
        'with' => ['stream' => 'php://stderr'],  
        'level' => 'error',  
    ],  
],
```

app/Shared/Traits/Auditable.php

```
<?php

namespace App\Shared\Traits;

use Illuminate\Support\Facades\Log;

trait Auditable
{
    public static function bootAuditable(): void
    {
        static::created(fn ($model) => static::logAudit('created', $model));
        static::updated(fn ($model) => static::logAudit('updated', $model));
        static::deleted(fn ($model) => static::logAudit('deleted', $model));
    }

    private static function logAudit(string $action, $model): void
    {
        Log::channel('audit')->info("[AUDIT] {$action}", [
            'model'      => class_basename($model),
            'id'         => $model->getKey(),
            'user_id'    => auth()->id(),
            'business_id'=> auth()->user()?->business_id,
            'changes'    => $model->getDirty(),
            'ip'         => request()->ip(),
        ]);
    }
}
```

STEP 17 — Setup Database Configuration Terbaik

config/database.php — MySQL Optimized

```
'mysql' => [
    'driver'           => 'mysql',
    'host'             => env('DB_HOST', '127.0.0.1'),
    'port'             => env('DB_PORT', '3306'),
    'database'         => env('DB_DATABASE', 'velora_db'),
    'username'         => env('DB_USERNAME', 'root'),
    'password'         => env('DB_PASSWORD', ''),
    'unix_socket'      => env('DB_SOCKET', ''),
    'charset'          => 'utf8mb4',
    'collation'        => 'utf8mb4_unicode_ci',
    'prefix'           => '',
    'prefix_indexes'   => true,
    'strict'           => true,
    'engine'           => 'InnoDB',
    'options'          => extension_loaded('pdo_mysql') ? array_filter([
        PDO::MYSQL_ATTR_SSL_CA           => env('MYSQL_ATTR_SSL_CA'),
        PDO::ATTR_PERSISTENT              => false,
        PDO::ATTR_EMULATE_PREPARES       => false,
    ]) : [],
],
```

Migration Best Practice

```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('products', function (Blueprint $table) {
            // Primary Key – UUID
            $table->uuid('id')->primary();

            // Multi-tenant isolation
            $table->uuid('business_id');
            $table->uuid('branch_id')->nullable();

            // Data columns
            $table->string('name', 255);
            $table->string('sku', 100)->unique();
            $table->string('barcode', 100)->nullable()->index();
            $table->text('description')->nullable();
            $table->bigInteger('selling_price'); // IDR cents/smallest unit
            $table->bigInteger('cost_price');
            $table->boolean('is_active')->default(true);
            $table->boolean('has_variants')->default(false);

            // Timestamps + soft delete
            $table->timestamps();
            $table->softDeletes();

            // Foreign Keys
            $table->foreign('business_id')->references('id')->on('businesses')->cascadeOnDelete();

            // Indexes (composite untuk query umum)
            $table->index(['business_id', 'is_active']);
            $table->index(['business_id', 'sku']);
            $table->index(['business_id', 'created_at']);
        });
    }
}
```

```
public function down(): void
{
    Schema::dropIfExists('products');
}
};
```

app/Shared/Traits/HasUuid.php

```
<?php

namespace App\Shared\Traits;

use Illuminate\Support\Str;

trait HasUuid
{
    public static function bootHasUuid(): void
    {
        static::creating(function ($model) {
            if (empty($model->{$model->getKeyName()})) {
                $model->{$model->getKeyName()} = (string) Str::uuid();
            }
        });
    }

    public function getIncrementing(): bool { return false; }
    public function getKeyType(): string { return 'string'; }
}
```


app/Shared/Traits/BelongsToBusiness.php

```
<?php

namespace App\Shared\Traits;

use Illuminate\Database\Eloquent\Builder;

trait BelongsToBusiness
{
    // Global scope – otomatis filter by business_id
    public static function bootBelongsToBusiness(): void
    {
        static::addGlobalScope('business', function (Builder $query) {
            if (auth()->check() && auth()->user()->business_id) {
                $query->where('business_id', auth()->user()->business_id);
            }
        });

        static::creating(function ($model) {
            if (auth()->check() && empty($model->business_id)) {
                $model->business_id = auth()->user()->business_id;
            }
        });
    }
}
```

STEP 18 — Setup Environment Production-Ready

.env.production

```
APP_NAME=VELORA
APP_ENV=production
APP_DEBUG=false
APP_KEY=base64:GENERATE_FRESH_KEY_HERE
APP_URL=https://api.velora.id
APP_TIMEZONE=Asia/Jakarta

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=velora_production
DB_USERNAME=velora_user
DB_PASSWORD=STRONG_PASSWORD_HERE

CACHE_STORE=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis

REDIS_HOST=127.0.0.1
REDIS_PASSWORD=REDIS_PASSWORD_HERE
REDIS_PORT=6379

LOG_CHANNEL=daily
LOG_LEVEL=error

SANCTUM_STATEFUL_DOMAINS=velora.id,app.velora.id

# Midtrans Production
MIDTRANS_SERVER_KEY=PROD_SERVER_KEY
MIDTRANS_CLIENT_KEY=PROD_CLIENT_KEY
MIDTRANS_IS_PRODUCTION=true

# OpenAI
OPENAI_API_KEY=sk-PRODUCTION_KEY
OPENAI_MODEL=gpt-4o

# Mail
MAIL_MAILER=smtp
```

```
MAIL_HOST=smtp.mailgun.org
MAIL_PORT=587
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=noreply@velora.id
```

Production Optimization Commands

```
# Optimize Laravel
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache
php artisan optimize

# Install dependencies (no dev)
composer install --optimize-autoloader --no-dev

# Run migrations
php artisan migrate --force

# Queue worker (pakai Supervisor)
php artisan queue:work redis --sleep=3 --tries=3 --max-time=3600
```

supervisord.conf — Queue Worker

```
[program:velora-worker]
command=php /var/www/velora-api/artisan queue:work redis --sleep=3 --tries=3 --max-time=3600
directory=/var/www/velora-api
user=www-data
numprocs=4
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
stderr_logfile=/var/log/velora-worker.err.log
stdout_logfile=/var/log/velora-worker.out.log
```

app/Providers/AppServiceProvider.php — Production Hardening

```
<?php
```

```
namespace App\Providers;
```

```
use Illuminate\Database\Eloquent\Model;
```

```
use Illuminate\Http\Resources\Json\JsonResource;
```

```
use Illuminate\Support\Facades\DB;
```

```
use Illuminate\SupportServiceProvider;
```

```
class AppServiceProvider extends ServiceProvider
```

```
{
```

```
    public function boot(): void
```

```
    {
```

```
        // Strict mode di development
```

```
        Model::shouldBeStrict(!app()->isProduction());
```

```
        // Tidak wrap resource dalam 'data' key (kita pakai ApiResponse sendiri)
```

```
        JsonResource::withoutWrapping();
```

```
        // Log slow queries (> 500ms) di production
```

```
        if (app()->isProduction()) {
```

```
            DB::whenQueryingForLongerThan(500, function () {
```

```
                \Illuminate\Support\Facades\Log::channel('api')->warning('Slow query detected',
```

```
                    'queries' => DB::getQueryLog(),
```

```
                ]);
```

```
            });
```

```
        }
```

```
    }
```

```
}
```

Summary: Perintah Install Lengkap

1. Create Project

```
composer create-project laravel/laravel velora-api "^13.0"  
cd velora-api
```

2. Install packages

```
composer require laravel/sanctum spatie/laravel-permission predis/predis
```

3. Publish & migrate

```
php artisan vendor:publish --provider="Laravel\Sanctum\SanctumServiceProvider"  
php artisan vendor:publish --provider="Spatie\Permission\PermissionServiceProvider"  
php artisan migrate
```

4. Generate key

```
php artisan key:generate
```

5. Autoload helpers

```
composer dump-autoload
```

6. Start server

```
php artisan serve
```

Folder Structure Final

```
app/  
├─ Console/Commands/  
├─ Exceptions/Handler.php  
├─ Modules/  
│   ├─ Auth/  
│   ├─ Tenant/  
│   ├─ Subscription/  
│   ├─ POS/  
│   ├─ Inventory/  
│   ├─ Sales/  
│   ├─ Purchasing/  
│   ├─ Finance/  
│   ├─ CRM/  
│   ├─ Employee/  
│   ├─ Notification/  
│   ├─ Report/  
│   ├─ Audit/  
│   └─ Setting/  
├─ Providers/  
│   ├─ AppServiceProvider.php  
│   ├─ RepositoryServiceProvider.php  
│   └─ ModuleServiceProvider.php  
└─ Shared/  
    ├─ Contracts/BaseRepositoryInterface.php  
    ├─ Controllers/BaseController.php  
    ├─ Exceptions/VeloraException.php  
    ├─ Helpers/  
    │   ├─ ApiResponse.php  
    │   ├─ MoneyHelper.php  
    │   ├─ CodeGenerator.php  
    │   └─ helpers.php  
    ├─ Middleware/  
    │   ├─ EnsureTenantScope.php  
    │   ├─ EnsureActiveSubscription.php  
    │   ├─ EnsureBranchAccess.php  
    │   └─ CheckFeatureLimit.php  
    ├─ Repositories/BaseRepository.php  
    ├─ Services/BaseService.php  
    └─ Traits/
```

- └─ HasUuid.php
- └─ BelongsToBusiness.php
- └─ BelongsToBranch.php
- └─ Auditable.php